

# Phaeton Capital

## SentimentCrowd

---

### Statistical & Mathematical Audit Report

Multi-Agent Deep Review  
5 Specialists + Devil's Advocate

Date: March 23, 2026  
Prepared by: Claude Opus 4.6 Multi-Agent Team  
Repository: [github.com/JafarAlbadri/phaeton\\_capital](https://github.com/JafarAlbadri/phaeton_capital)

# Table of Contents

---

- 1. Executive Summary
- 2. Methodology
- 3. Architecture Overview
- 4. Critical Findings (P0)
- 5. High Priority Findings (P1)
- 6. Statistical Methodology Issues
- 7. Data Architecture Issues
- 8. ML/AI Pipeline Issues
- 9. Numerical Stability Issues
- 10. Devil's Advocate: False Positives
- 11. Devil's Advocate: Missed Issues (False Negatives)
- 12. Recommended Action Plan
- 13. Proposed Statistical Methods

# 1. Executive Summary

This report presents findings from a comprehensive multi-agent statistical and mathematical audit of the Phaeton Capital / SentimentCrowd codebase. The review was conducted by five specialist agents (Statistician, Quantitative Financial Analyst, Data Architect, ML/AI Engineer, Systems Programmer) followed by a Devil's Advocate review to identify false positives and missed issues.

The primary goal was to ensure statistical significance across all mathematical computations, identify flawed assumptions, and recommend rigorous methods for a production-grade sentiment analysis platform.

## Key Metrics

| Metric                                  | Value                  |
|-----------------------------------------|------------------------|
| Files analyzed                          | 12                     |
| Specialist agents                       | 5 + 1 Devil's Advocate |
| Critical findings (P0)                  | 3                      |
| High priority findings (P1)             | 5                      |
| Medium priority findings (P2)           | 6                      |
| False positives identified              | 4                      |
| Missed issues found by Devil's Advocate | 10                     |

VERDICT: The system presents AI-generated sentiment scores as if they were validated market data. The trust scoring mechanism operates on fabricated (random) data, the statistical distribution model is mathematically inappropriate, and there is no evaluation pipeline to validate AI accuracy. A user making investment decisions based on this dashboard has no statistical basis to trust the numbers.

## 2. Methodology

---

The audit was performed by six specialized AI agents working in parallel:

| Agent                | Focus Area                                                                          |
|----------------------|-------------------------------------------------------------------------------------|
| Statistician         | Gaussian distributions, significance testing, Bayesian inference, bootstrap methods |
| Quantitative Analyst | Financial calculations, trust scoring, signal pipeline, fat-tail distributions      |
| Data Architect       | Data model, bias analysis, sampling bias, survivorship bias, pipeline architecture  |
| ML/AI Engineer       | AI pipeline, prompt engineering, model evaluation, calibration, drift detection     |
| Systems Programmer   | Numerical stability, floating point issues, edge cases, algorithmic correctness     |
| Devil's Advocate     | False positive screening, false negative identification, practical risk assessment  |

### 3. Architecture Overview

---

Phaeton Capital / SentimentCrowd is a monorepo sentiment analysis platform consisting of:

- Next.js 15 web dashboard (apps/web) - port 3000
- Bun worker service (apps/worker) - port 8080, scrapes Reddit/Google News/Yahoo Finance
- PostgreSQL 15 database with Prisma ORM (packages/db)
- Google Gemini AI (gemini-2.5-flash) for sentiment classification
- Python subprocess (yfinance) for fundamental financial data
- Docker Compose for orchestration

Data flow: Worker scrapes social media/news every 15 minutes -> AI labels sentiment -> stores in PostgreSQL -> Dashboard reads and visualizes with Gaussian distribution curve.

## 4. Critical Findings (P0)

### 4.1 Trust Score Based on Math.random()

SEVERITY: CRITICAL. The entire anti-manipulation pipeline is rendered meaningless because `author_karma` and `account_age_days` for Reddit posts are generated using `Math.random()`, not fetched from the Reddit API.

```
// scraper.ts:50-51
const pseudoKarma = Math.floor(Math.random() * 5000);
const pseudoAgeDays = Math.floor(Math.random() * 300);
```

`computeTrustScore()` requires `karma > 100` and `ageDays > 30`. With random values:  $P(\text{karma} > 100) = 98\%$ ,  $P(\text{age} > 30) = 90\%$ . Combined: ~88% of posts pass the trust filter regardless of whether they are bots or legitimate users. The remaining ~12% are randomly excluded and their sentiment is set to 0, introducing artificial neutral bias.

Confirmed by: Quantitative Analyst, Data Architect, Systems Programmer, Devil's Advocate.

### 4.2 Unauthenticated /api/scrape Endpoint

SEVERITY: CRITICAL (discovered by Devil's Advocate - missed by all 5 specialists). The `/api/scrape` endpoint accepts POST requests without any authentication or rate limiting. Each scrape triggers Gemini API calls (which cost money), database writes, and Python subprocesses. An attacker can trigger unlimited scrapes, exhausting API credits.

### 4.3 Overly Aggressive AI Fallback Condition

SEVERITY: CRITICAL. The fallback to `gemini-1.5-flash` triggers on ALL HTTP errors, not just 429 rate limits.

```
// ai.ts:88 - the !!err?.status catches ALL errors with status
if (err?.message?.includes('429') || err?.status === 429
    || err?.message?.includes('Quota') || !!err?.status) {
```

This means authentication errors (401), bad requests (400), and server errors (500) all silently fall back to a different model with different calibration. No logging records which model generated each result, making the database non-reproducible.

## 5. High Priority Findings (P1)

---

### 5.1 Missing database index on ticker in Sentiment table

schema.prisma - Sentiment model

Case-insensitive findMany queries on ticker without index cause sequential scans. Performance degrades exponentially as data grows (hundreds of new posts every 15 minutes).

### 5.2 Untrusted posts set to sentiment=0 instead of excluded

index.ts:197

finalSentiment = isTrusted ? aiResult.sentiment : 0. Posts with sentiment=0 are saved to the database and included in aggregation, artificially pulling the Gaussian mean toward neutral. They should be excluded entirely or flagged with an 'excluded' field.

### 5.3 Unhandled Promise in worker HTTP handler

index.ts:248

processPosts() is called fire-and-forget without await or .catch(). Unhandled promise rejections can crash the Bun worker process.

### 5.4 Race condition on concurrent /trigger calls

index.ts:248

No mutex or lock prevents concurrent processPosts() executions. Multiple /trigger calls or cron overlap results in duplicate API calls to Gemini (wasting credits) and potential data corruption.

### 5.5 No AI model version logged in database

ai.ts, schema.prisma

The database schema has no field for which AI model produced each sentiment score. When fallback is used, data from different models with different calibrations is mixed without traceability.

## 6. Statistical Methodology Issues

---

### 6.1 Gaussian Distribution on Bounded Data

The dashboard uses a Gaussian PDF to model sentiment values bounded to  $[-1, 1]$ . A normal distribution has support on  $(-\infty, +\infty)$  and leaks probability mass outside the valid range. While the Devil's Advocate assessed this as ~80% false positive (acceptable for visualization), the following sub-issues are real:

- No normality test (Shapiro-Wilk, Kolmogorov-Smirnov) is performed
- Sentiment data is likely bimodal (clusters at bullish/bearish extremes)
- The truncated PDF is not renormalized - integral over  $[-1, 1] \neq 1$
- stdDev fallback to 0.2 is arbitrary and undocumented

### 6.2 No Statistical Significance Testing

The dashboard labels sentiment as 'Bullish' (mean  $> 0.2$ ) or 'Bearish' (mean  $< -0.2$ ) without any significance test. These arbitrary thresholds have no statistical basis. Required:

- One-sample t-test: show Bullish/Bearish ONLY when  $p < 0.05$
- Bootstrap 95% confidence interval around the mean
- Effect size (Cohen's  $d$ ) relative to noise level
- Power analysis: with  $n \sim 100-250$  posts, can we detect meaningful shifts?

### 6.3 Independence Assumption Violated

Standard statistical tests assume independent observations. Reddit posts violate this: posts in the same thread are correlated, bandwagon effects exist, same authors post multiple times, and different news sources report the same event. No corrections are applied (cluster-robust standard errors, HAC estimators).

### 6.4 No Confidence Intervals

Market Consensus is displayed as a point estimate without uncertainty. A mean of 15% based on 10 posts should look completely different from the same mean based on 1000 posts. At minimum: display mean  $\pm 1.96 \cdot SE$  with sample size  $n$ .



## 7. Data Architecture Issues

---

### 7.1 Sampling Bias

- Reddit search returns max 100 posts per request with no pagination
- Only 3 English-language sources out of hundreds of possible platforms
- Google News URL hardcoded to hl=en-US&gl=US - blind to non-English sentiment
- 15-minute scrape interval can miss intraday sentiment spikes

### 7.2 Survivorship Bias

- Deleted Reddit posts are never captured
- Shadowbanned/banned accounts invisible in search results
- Moderator-removed posts (aggressive in WSB) not captured
- Sample systematically biased toward posts that 'survived' moderation

### 7.3 Volume Bias

- No normalization per time window - 200 posts one day vs 10 posts next day weighted equally
- No normalization per source - Reddit dominates despite potentially lower quality
- take: 250 represents variable time periods depending on activity level

### 7.4 Missing Provenance

- No 'source' column - impossible to distinguish Reddit from news data in retrospect
- No scrape\_batch\_id - no way to trace data lineage
- No collection\_timestamp separate from post\_timestamp
- Fabricated metadata (random karma/age) stored as if real

### 7.5 Destructive Insider Data Handling

index.ts:82-84 uses deleteMany + createMany without a transaction wrapper. If createMany fails after deleteMany, all insider data is permanently lost.

## 8. ML/AI Pipeline Issues

---

### 8.1 No Ground Truth or Evaluation

The AI model labels sentiment without any validation against actual market outcomes. No precision, recall, F1, confusion matrix, or ROC-AUC metrics exist. While the Devil's Advocate noted this is acceptable for an MVP, it means the system is an unvalidated hypothesis generator.

### 8.2 Non-Deterministic Output

Temperature is never set in the Gemini API call (ai.ts:47-59). The same post can produce different sentiment scores on different runs. Setting temperature=0 would ensure reproducibility.

### 8.3 Circular Confidence Score

The model generates its own confidence (ai.ts:37), which is fundamentally flawed from an ML perspective. LLMs are notoriously overconfident and poorly calibrated. Confidence 0.9 from Gemini does not mean 90% probability of correctness. No calibration curve has been validated.

### 8.4 Batch Context Contamination

Posts are batched by token count (max 4000 tokens, max 25 posts). Each batch is an isolated API call. A bearish post in a predominantly bullish batch may receive anchoring bias from surrounding context.

### 8.5 Confidence Not Used in Aggregation

The confidence field is stored in the database but never used in downstream calculations. All posts are weighted equally regardless of confidence level.

## 9. Numerical Stability Issues

### 9.1 Confirmed Issues

| Operation              | Status                                    |
|------------------------|-------------------------------------------|
| Gaussian PDF formula   | CORRECT - matches mathematical definition |
| Mean calculation       | CORRECT - empty array check present       |
| Variance calculation   | CORRECT - uses Bessel's correction (n-1)  |
| SHA-256 hash           | CORRECT - deterministic for same input    |
| Date/time handling     | CORRECT - proper UTC conversion           |
| safe_float() in Python | CORRECT - handles None, NaN, Inf          |

### 9.2 Issues Found

#### shares\_out fallback to 1

yfinance\_fetcher.py:43 - if both sharesOutstanding and impliedSharesOutstanding are missing, shares\_out defaults to 1. This produces absurd EPS values (e.g., billions per share). Should return None instead.

#### Token estimation inaccuracy

index.ts:153 estimates 1 token per 4 characters. Unicode/emoji-heavy text can be 1 token per 1-2 characters, causing batches to exceed MAX\_TOKENS\_PER\_BATCH.

## 10. Devil's Advocate: False Positives

The following findings from the specialist agents were assessed as partially or fully overstated:

| Finding                              | FP % | Assessment                                                                                |
|--------------------------------------|------|-------------------------------------------------------------------------------------------|
| Gaussian PDF 'statistically invalid' | ~80% | Used only for visualization, not inference. KDE/Beta-distribution is academically correct |
| 'No ground truth' as a blocker       | ~60% | Correct as a future improvement, but unreasonable as a P0 requirement for an MVP prod     |
| Division by zero at RPM_LIMIT=0      | ~90% | Default is 4, configured in docker-compose.yml and .env.example. No sensible user sets    |
| NaN propagation corrupts Gaussian    | ~50% | PostgreSQL can store NaN in float columns, but Gemini practically never returns NaN. P    |

## 11. Devil's Advocate: Missed Issues

The following problems were missed by all 5 specialist agents and discovered by the Devil's Advocate:

| Issue                        | Severity | Description                                                             |
|------------------------------|----------|-------------------------------------------------------------------------|
| Unauthenticated /api/scrape  | HIGH     | No auth or rate-limiting. Anyone can trigger costly Gemini API calls.   |
| Race condition on /trigger   | MEDIUM   | No mutex - concurrent calls waste API credits and may corrupt data.     |
| Missing ticker index         | MEDIUM   | Sequential scan on case-insensitive queries. Degrades with data growth. |
| Unhandled Promise            | MEDIUM   | processPosts() fire-and-forget can crash the worker on errors.          |
| Cheerio imported but unused  | LOW      | Dead import in fundamentals.ts.                                         |
| Playwright dependency unused | LOW      | ~100MB unused dependency in Docker image.                               |
| Hardcoded Docker path        | LOW      | /app/apps/worker/... breaks local development.                          |
| yahoo-finance2 in web pkg    | LOW      | Unused dependency in frontend package.json.                             |
| BigInt serialization risk    | LOW      | Safe now but design weakness for large values.                          |
| Prisma connection pool       | LOW-MED  | Default 5 connections, hundreds of queries per run.                     |

## 12. Recommended Action Plan

---

### Phase 1 - Critical (breaks fundamental validity)

- Fix Math.random() trust score - implement real Reddit OAuth API or remove trust filter entirely
- Add authentication + rate-limiting on /api/scrape endpoint
- Fix AI fallback condition - only trigger on 429, not all HTTP errors
- Add temperature: 0 to Gemini API calls for reproducibility
- Add database index on ticker in Sentiment table

### Phase 2 - High (data quality & rigor)

- Add source, ai\_model, scrape\_batch\_id columns to Sentiment schema
- Range-validate AI output: sentiment in [-1,1], confidence in [0,1]
- Exclude untrusted posts instead of setting sentiment=0
- Await/catch processPosts() in HTTP handler
- Add mutex against concurrent scrapes
- Wrap destructive insider data operations in transactions

### Phase 3 - Statistical rigor

- Bootstrap 95% confidence intervals around sentiment mean
- One-sample t-test: show Bullish/Bearish ONLY at  $p < 0.05$
- Confidence-weighted sentiment aggregation
- Exponential decay for temporal weighting (48h half-life)
- Display sample size (n) alongside all statistics

### Phase 4 - Advanced (quantitative edge)

- Bayesian inference with Beta prior for incremental posterior updates
- Kernel Density Estimation (KDE) instead of parametric Gaussian
- Granger causality test: does sentiment actually lead price?
- Hidden Markov Model for regime detection (bull/bear/sideways)
- Inter-rater reliability: run same posts through 3+ models (Krippendorff's alpha)
- Kelly Criterion for position sizing based on sentiment edge
- Hurst exponent for momentum vs mean-reversion classification
- Monte Carlo stress tests on sentiment signals
- Sector-adjusted Z-scores for financial metrics

### Phase 5 - Cleanup

- Remove unused dependencies: cheerio, playwright, yahoo-finance2
- Make Python path configurable (not hardcoded Docker path)

- Add NaN filter before sentiment calculations

## 13. Proposed Statistical Methods

---

### 13.1 Beta Distribution (replaces Gaussian)

Sentiment  $[-1,1]$  transformed to  $[0,1]$  via  $(x+1)/2$ , modeled with  $\text{Beta}(\alpha, \beta)$ . Method of Moments estimation:  $\alpha = \text{mean} * C$ ,  $\beta = (1-\text{mean}) * C$ , where  $C = (\text{mean}*(1-\text{mean})/\text{variance}) - 1$ .

### 13.2 Bootstrap Confidence Intervals

Resample  $n$  observations with replacement 10,000 times. Compute mean for each bootstrap sample. Sort means and take 2.5th and 97.5th percentiles as 95% CI bounds. Display as: 'Market Consensus: 34% Bullish [95% CI: 21%-47%]'

### 13.3 Kernel Density Estimation (KDE)

Non-parametric density estimation using Gaussian kernels with Silverman's rule-of-thumb bandwidth:  $h = 1.06 * \sigma * n^{(-1/5)}$ . Makes no assumptions about underlying distribution shape. Captures multimodality (bullish/bearish clusters) that Gaussian misses.

### 13.4 Logistic Trust Function

Replace binary trust score with continuous sigmoid:  $\text{trustWeight} = \text{sigmoid}(\text{karma}, 500, 0.01) * \text{sigmoid}(\text{ageDays}, 60, 0.05)$ . Produces values in  $[0, 1]$  with gradual transition instead of hard cutoff.

### 13.5 Exponential Decay Weighting

$\text{Weight} = \exp(-\ln(2) * \text{hoursAgo} / \text{halfLife})$ . With 48h half-life: posts from 2 days ago weigh 50%, 4 days = 25%, 1 week = 6.25%. Ensures recent sentiment dominates without discarding older data entirely.

### 13.6 Signal-to-Noise Ratio (SNR)

$\text{SNR} = |\text{mean}(\text{sentiment})| / \text{stddev}(\text{sentiment})$ .  $\text{SNR} < 1$  means noise dominates and the signal is unreliable. Display SNR alongside sentiment to indicate confidence in the directional signal.

### 13.7 Granger Causality

Test whether lagged sentiment values improve prediction of price changes beyond what lagged prices alone provide. Requires time-aligned series at equal frequency. F-test comparing restricted (price-only) vs unrestricted (price + sentiment) regression. If  $p < 0.05$ , sentiment Granger-causes price movement.



## End of Report

Prepared by Claude Opus 4.6 Multi-Agent Team

Date: March 23, 2026

6 agents | 12 files | 24 findings

*This report was generated automatically. All findings should be verified  
against the current state of the codebase before taking action.*